

Creation of a NoSQL Database for Credit Card Fraud Detection

Technical documentation — DBMS project

Ulysse Auplat

Vijay Singh

June 2026

1 Introduction

This project builds a NoSQL database for credit card fraud detection, populated with synthetic data produced by the transaction simulator of the handbook *Machine Learning for Credit Card Fraud Detection*. Among the systems seen in the course we chose **Neo4j**: the main operations of the workload (customers sharing terminals, co-customer network of degree k , frequent collaborators) are path traversals, which a graph database answers naturally, while a relational or document model would need repeated joins.

Why Neo4j instead of MongoDB?

Why would a document-oriented database like MongoDB be insufficient for this workload? A document model stores transactions as embedded objects within customer or terminal documents, making multi-hop traversals (e.g., customer $\rightarrow$ terminal $\rightarrow$ customer $\rightarrow$ terminal) require multiple document lookups and application-level joins. In contrast, Neo4j's graph structure materialises these relationships explicitly, allowing the query engine to traverse edges in a single operation. For operations like finding co-customers of degree 3 (operation c), a document database would require 2–3 separate collection queries and explicit join logic in the application, whereas Neo4j expresses the same logic in a declarative Cypher query with graphical efficiency.

2 Dataset generation

The datasets were generated with the simulator notebook (`SimulatedDataset.ipynb`). Customers and terminals are placed on a 100×100 grid and each customer only uses the terminals within a radius $r = 5$ of its position. We kept the population fixed (2 000 customers, 300 terminals, satisfying the minimum of the assignment) and increased the number of simulated days to reach the required sizes.

Dataset	Customers	Terminals	Days	Transactions	CSV size
50 MB	2 000	300	300	1 015 882	52 MB
100 MB	2 000	300	600	2 031 891	105 MB
200 MB	2 000	300	1200	4 061 394	213 MB

Table 1: The three generated datasets (start date 2018-04-01). Each dataset consists of a customer, a terminal and a transaction CSV file.

3 Conceptual model

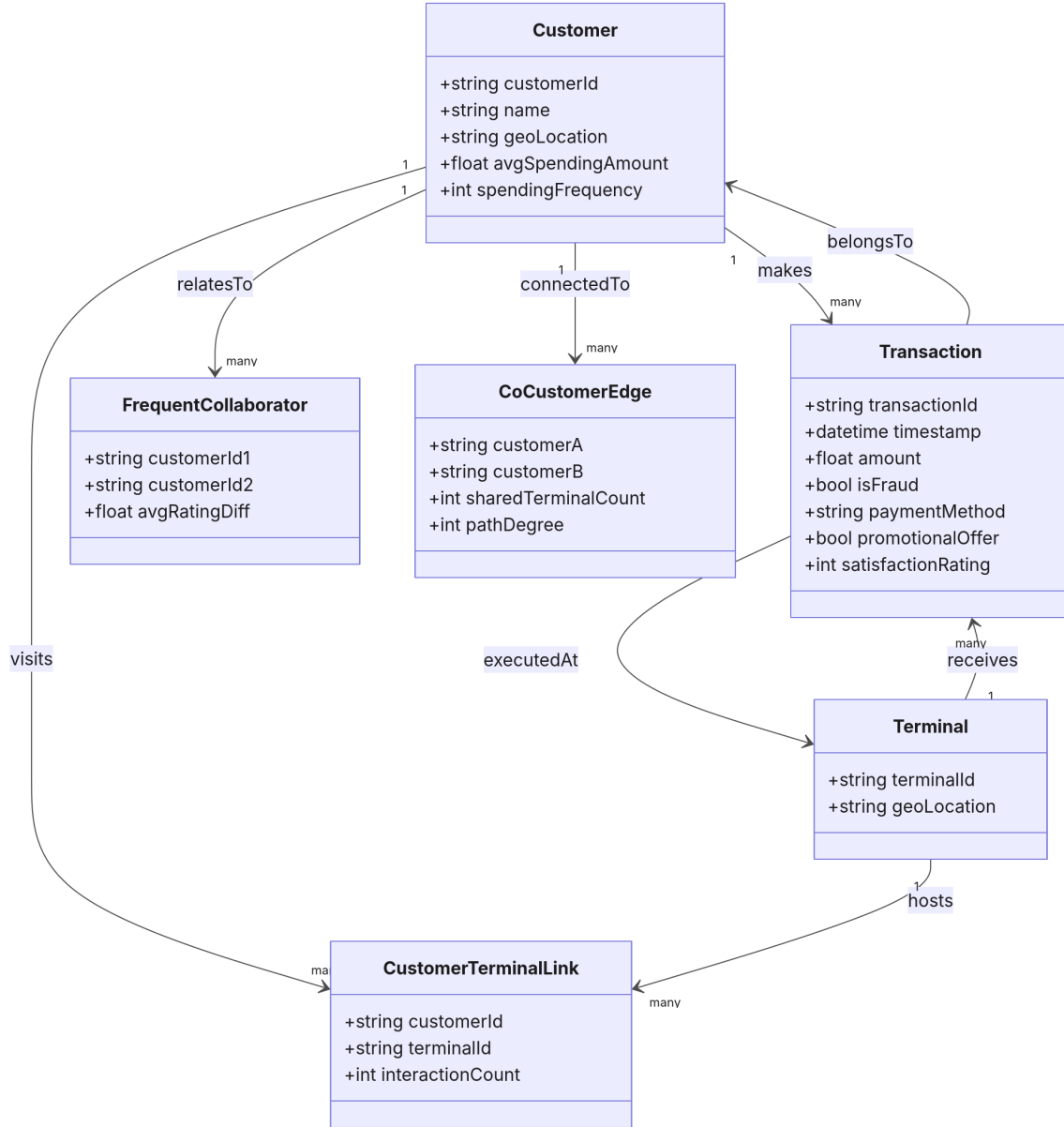


Figure 1: UML class diagram, including the extensions of operation d.

Customer is identified by **customerId** and carries its location and spending profile (average amount, spending frequency). **Terminal** only carries its identifier and location. **Transaction** is a *weak class*: it has no autonomous existence and is identified by the customer who made it, the terminal that received it and the timestamp. Each transaction belongs to exactly one customer and one terminal (1–many associations **makes** and **receives**). **CustomerTerminalLink** materialises the “customer uses terminal” association stored in the customer profile.

The diagram already includes the modifications required by operation d: the attributes **paymentMethod**, **promotionalOffer** and **satisfactionRating** on **Transaction**, the derived class **FrequentCollaborator** (operation d.ii) and the derived class **CoCustomerEdge**, which represents the co-customer relation traversed by operation c.

Assumptions and constraints:

- a location is a point in a 100×100 grid;

- a customer can only use terminals within radius r of its location; a customer with no terminal in its radius cannot make any transaction;
- `satisfactionRating` is an integer in $[1, 5]$ and `paymentMethod` is one of {Credit Card, Mobile Payment, PayPal}; both are assigned randomly after loading, as allowed by the assignment;
- `FrequentCollaborator` is symmetric and irreflexive: each pair is stored once, with $c_1 < c_2$.

4 Logical data model

Element	Type	Properties
Customer	node	id, x, y, mean_amount, std_amount, mean_tx_per_day, nb_terminals
Terminal	node	id, x, y
MADE_TRANSACTION	relationship (Customer → Terminal)	tx_id, amount, tx_datetime, tx_time_days, tx_time_seconds, paymentMethod, promotionalOffer, satisfactionRating, isOutlier
FREQUENT_COLLABORATOR	relationship (Customer → Customer)	terminalId, avgRatingDiff

Table 2: Logical graph model.

Motivations:

- **Transactions as relationships, not as nodes.** A transaction only connects one customer to one terminal, so it is stored as a `MADE_TRANSACTION` relationship with the transaction data as properties. This halves the hops of every traversal used by operations a and c (the pattern is directly (*Customer*) → (*Terminal*)) and avoids creating millions of extra nodes. The trade-off is that relationship properties are not indexed; in particular the timestamp is a string parsed with `datetime()` at query time, which costs in operations b and e.
- **Indexes on `Customer.id` and `Terminal.id`,** created before loading: the transaction load does two `MATCH` by id per row, and operations starting from a given customer (operation c) become index seeks. Without these indexes, each load operation would require a full scan of all customers and terminals; with indexes, Neo4j performs fast $O(\log n)$ lookups that are essential for loading millions of transactions efficiently.
- **Relationships as first-class entities:** By storing transactions as `MADE_TRANSACTION` relationships rather than intermediate nodes, we reduce traversal depth and enable direct 2-hop patterns (Customer → Terminal) which operations a and c exploit. The `FREQUENT_COLLABORATOR` relationships (operation d.ii) further optimize path queries by pre-computing and materialising co-customer links.
- **Materialised derived data:** the `isOutlier` flag (used by operation e) and the `FREQUENT_COLLABORATOR` relationship (operation d.ii) are computed once and stored, so they are directly queryable afterwards, avoiding repeated computation.

5 Loading the datasets

The load is done with Cypher `LOAD CSV` (the CSV files are first copied into the `import` folder of the instance), in this order:

1. Indexes.

```
CREATE INDEX terminal_id_index FOR (t:Terminal) ON (t.id);
CREATE INDEX customer_id_index FOR (c:Customer) ON (c.id);
```

2. Customer and terminal nodes.

```
LOAD CSV WITH HEADERS FROM 'file:///customer_100MB.csv' AS row
CREATE (:Customer {
  id: toInteger(row.CUSTOMER_ID),
  x: toFloat(row.x_customer_id),
  y: toFloat(row.y_customer_id),
  mean_amount: toFloat(row.mean_amount),
  std_amount: toFloat(row.std_amount),
  mean_tx_per_day: toFloat(row.mean_nb_tx_per_day),
  nb_terminals: toInteger(row.nb_terminals)
});

LOAD CSV WITH HEADERS FROM 'file:///terminal_100MB.csv' AS row
CREATE (:Terminal {
  id: toInteger(row.TERMINAL_ID),
  x: toFloat(row.x_terminal_id),
  y: toFloat(row.y_terminal_id)
});
```

3. Transactions. The big file is loaded in batches of 10 000 rows (CALL {} IN TRANSACTIONS), which keeps memory usage constant; each row finds its endpoints through the indexes created in step 1. The batching strategy is critical: without it, loading millions of transactions would exhaust available memory and fail.

```
LOAD CSV WITH HEADERS FROM 'file:///transactions_100MB.csv' AS row
CALL (row) {
  MATCH (c:Customer {id: toInteger(row.CUSTOMER_ID)})
  MATCH (t:Terminal {id: toInteger(row.TERMINAL_ID)})
  CREATE (c)-[:MADE_TRANSACTION {
    tx_id: toInteger(row.TRANSACTION_ID),
    amount: toFloat(row.TX_AMOUNT)
  }]->(t)
} IN TRANSACTIONS OF 10000 ROWS;
```

A second batched pass sets the temporal properties (tx_datetime, tx_time_days, tx_time_seconds) on each relationship, matched by tx_id. After the load, a verification script counts the loaded entities and reports the import timings (Figure ??).

Your 100MB graph is loaded correctly.

Current counts	
Entity	Count
Customers	2,000
Terminals	300
Transactions	2,031,891

Import timings	
Operation	Time
Customer import	212 ms
Terminal import	31 ms
Transaction import	12.4 s

Figure 2: Load verification, 100 MB dataset: the counts match the CSV files exactly. The time is dominated by the transaction import, as expected.

6 Implementation of the operations

All operations are written in Cypher and were run on the three database instances.

6.1 Operation a – customers with shared terminals

For each customer M , find the customers N that used at least four of the same terminals and whose total transaction count differs by at most two.

```
MATCH (m:Customer)-[:MADE_TRANSACTION]->(t:Terminal)
      <-[:MADE_TRANSACTION]-(n:Customer)
WHERE m.id < n.id
WITH m, n, COUNT(DISTINCT t) AS sharedTerminals
WHERE sharedTerminals >= 4
MATCH (m)-[rm:MADE_TRANSACTION]->()
WITH m, n, sharedTerminals, COUNT(rm) AS txCountM
MATCH (n)-[rn:MADE_TRANSACTION]->()
WITH m, n, sharedTerminals, txCountM, COUNT(rn) AS txCountN
WHERE abs(txCountM - txCountN) <= 2
RETURN m.id AS CustomerM, n.id AS CustomerN,
       sharedTerminals, txCountM, txCountN
```

The 2-hop pattern enumerates the pairs of customers connected through a common terminal; $m.id < n.id$ considers each pair once and $COUNT(DISTINCT t)$ counts the *distinct* shared terminals. The shared-terminal filter is applied *before* counting the total transactions of the two customers, so the expensive counts are only computed for the few surviving pairs.

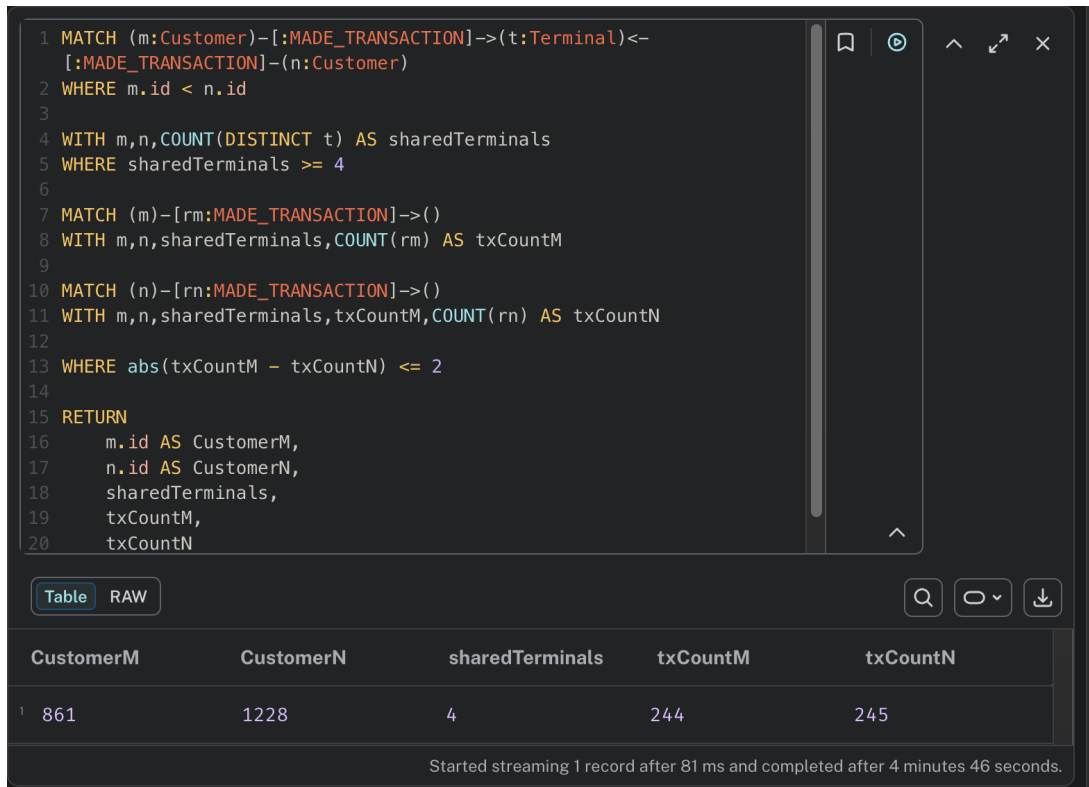


Figure 3: Operation a, 50 MB: one qualifying pair (861–1228, 4 shared terminals, 244 vs. 245 transactions), in 4 min 46 s.

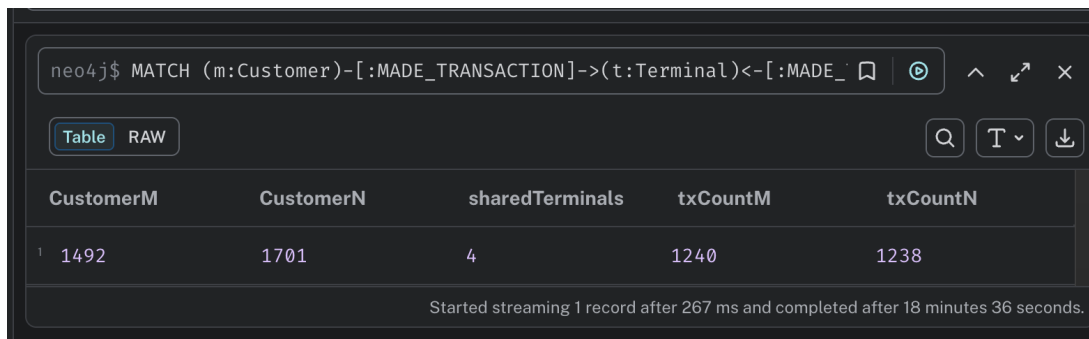


Figure 4: Operation a, 100 MB: pair 1492–1701, in 18 min 36 s.

```

1 MATCH (m:Customer)-[:MADE_TRANSACTION]->(t:Terminal)-
  [:MADE_TRANSACTION]-
2 (n:Customer)
3 WHERE m.id < n.id
4 WITH m,n,COUNT(DISTINCT t) AS sharedTerminals
5 WHERE sharedTerminals >= 4
6 MATCH (m)-[rm:MADE_TRANSACTION]->()
7 WITH m,n,sharedTerminals,COUNT(rm) AS txCountM
8 MATCH (n)-[rn:MADE_TRANSACTION]->()
9 WITH m,n,sharedTerminals,txCountM,COUNT(rn) AS txCountN
10 WHERE abs(txCountM - txCountN) <= 2
11 RETURN
12 m.id AS CustomerM,
13 n.id AS CustomerN,
14 sharedTerminals,
15 txCountM,
16 txCountN

```

CustomerM	CustomerN	sharedTerminals	txCountM	txCountN
591	1317	4	1158	1158

Started streaming 1 record after 1 ms and completed after 1 hour 30 minutes.

Figure 5: Operation a, 200 MB: pair 591–1317, in 1 h 30 min.

6.2 Operation b – transactions above 130% of the previous quarter median (outlier detection)

For each terminal, the median transaction amount of the previous quarter (Q4 2018 in our data) is computed, and the transactions of the current quarter (Q1 2019) whose amount exceeds 130% of that median are returned as potential outliers. This query removes outliers by defining a per-terminal threshold based on historical spending patterns. In the setup phase, we extract Q4 2018 transactions with a filter on the quarter and year properties before computing the median, ensuring only relevant historical data is used. The 130% threshold is applied to identify anomalous transactions that warrant investigation.

```

MATCH (t:Terminal)
CALL {
  WITH t
  MATCH (t)-[rPrev:MADE_TRANSACTION]-()
  WHERE datetime(replace(rPrev.tx_datetime,' ','T')).year = 2018
    AND datetime(replace(rPrev.tx_datetime,' ','T')).quarter = 4
  WITH rPrev.amount AS amount
  ORDER BY amount
  WITH collect(amount) AS amounts
  WITH amounts, size(amounts) AS n
  RETURN CASE
    WHEN n % 2 = 1 THEN amounts[toInteger(n/2)]
    ELSE (amounts[toInteger(n/2)-1] + amounts[toInteger(n/2)]) / 2.0
  END AS medianQ4
}
MATCH (t)-[rCurrent:MADE_TRANSACTION]-()
WHERE datetime(replace(rCurrent.tx_datetime,' ','T')).year = 2019
  AND datetime(replace(rCurrent.tx_datetime,' ','T')).quarter = 1
  AND rCurrent.amount > medianQ4 * 1.3
RETURN t.id AS terminalId, rCurrent.tx_id AS transactionId,
  rCurrent.amount AS amount, medianQ4,
  medianQ4 * 1.3 AS threshold
ORDER BY amount DESC
LIMIT 100;

```

Cypher has no median aggregator, so the Q4 amounts of each terminal are sorted, collected and the middle element is taken. Due to memory limitations, this terminal-by-terminal computation with a CALL subquery was adopted instead of a global aggregation approach. The

terminal-by-terminal approach significantly reduces the memory footprint by processing one terminal at a time rather than collecting all transactions globally. A second variant (delivered with the scripts) replaces the final RETURN with SET rCurrent.isOutlier = true to mark the potential outliers, which is what operation e reads. This outlier-marking pass is part of the setup phase and must complete successfully before operation e can report accurate outlier statistics.

The query executed successfully on the 50 MB dataset but exhausted the transaction memory on the 100 MB and 200 MB datasets (Figure ??).

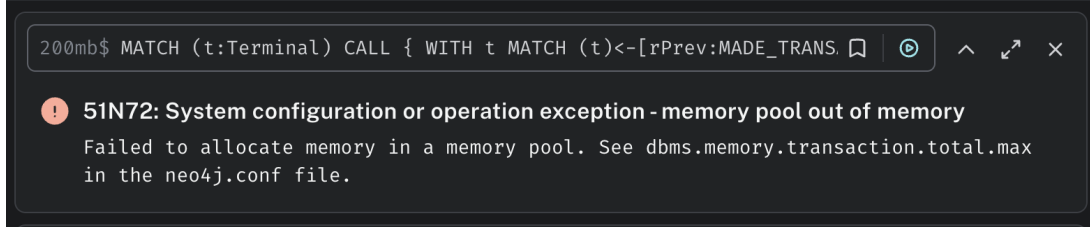


Figure 6: Operation b, 200 MB: error 51N72, memory pool out of memory (the same error occurs at 100 MB). The per-terminal median collects and sorts large lists, which exceeds the configured transaction memory.

6.3 Operation c – co-customer network of degree 3

$CN_k(u)$ contains the customers reachable from u through a path $u_1 - t_1 - u_2 - \dots - t_{k-1} - u_k$ with pairwise distinct customers. As suggested by the assignment we computed $CN_3(u)$. The direct formulation matches $(u_1) \rightarrow (t_1) \leftarrow (u_2) \rightarrow (t_2) \leftarrow (u_3)$, but every pair of customers is connected by many parallel transactions, so the number of matched paths explodes. The optimised version deduplicates with WITH DISTINCT after *each* step, so every expansion starts from a set of distinct nodes:

```
MATCH (u1:Customer {id: 0})-[:MADE_TRANSACTION]->(t1:Terminal)
WITH DISTINCT u1, t1
MATCH (t1)-[:MADE_TRANSACTION]-(u2:Customer)
WHERE u2 <> u1
WITH DISTINCT u1, u2
MATCH (u2)-[:MADE_TRANSACTION]->(t2:Terminal)
WITH DISTINCT u1, u2, t2
MATCH (t2)-[:MADE_TRANSACTION]-(u3:Customer)
WHERE u3 <> u1 AND u3 <> u2
RETURN DISTINCT u3.id
LIMIT 50;
```


16 LIMIT 50;

Table RAW

u3.id

1	134
2	154
3	476
4	479
5	657
6	764
7	959
8	1018
9	1376
10	9

Started streaming 50 records after 125 ms and completed after 159 ms.

> 03N96: Code generation failed

Figure 7: CN_3 (customer 0), 50 MB: 50 members returned in 159 ms. (The 03N96 message is only a notification: Neo4j fell back to the interpreted runtime; the result is unaffected.)

200mb\$ MATCH (u1:Customer {id: 0})-[:MADE_TRANSACTION]->(t1:Terminal)

Table RAW

u3.id

1	134
2	154
3	476
4	479
5	657
6	764
7	959
8	1018
9	1376
10	9

Started streaming 50 records after 278 ms and completed after 828 ms.

> 03N96: Code generation failed

Figure 8: CN_3 (customer 0), 200 MB: same result in 828 ms. Thanks to the stepwise deduplication the cost depends on the number of distinct customers and terminals, not on the number of transactions.

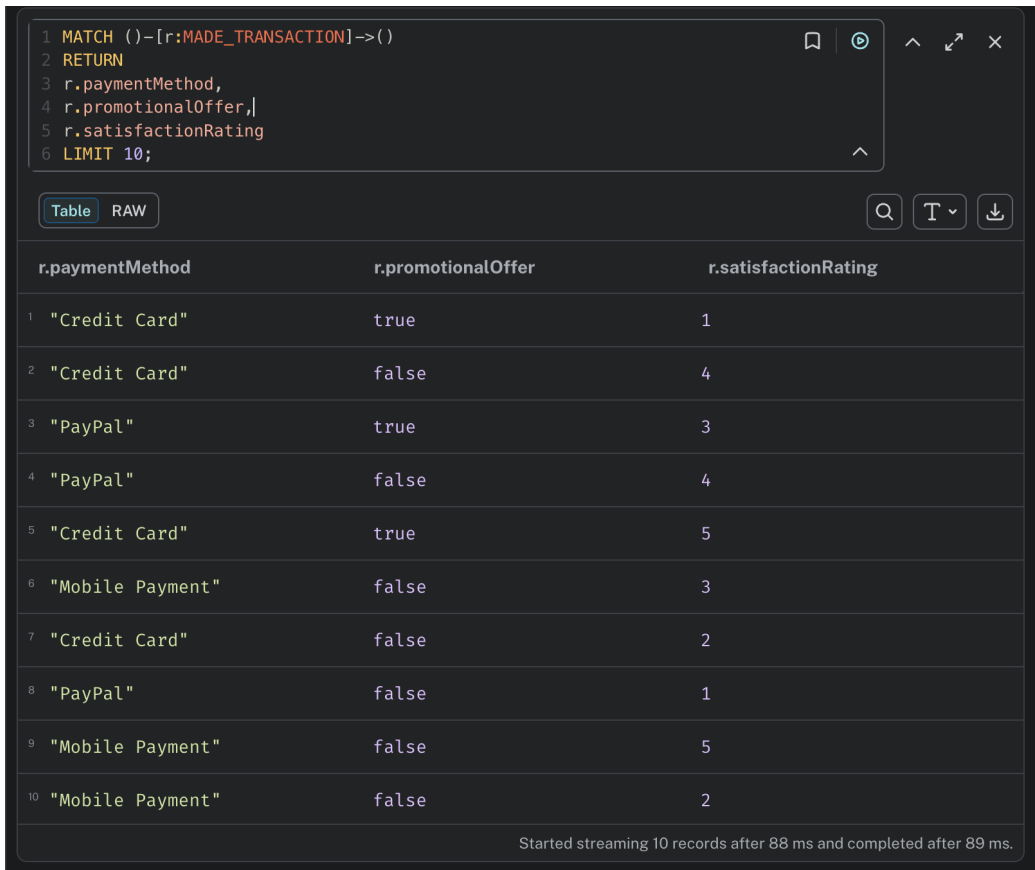
6.4 Operation d – extension of the logical model

This operation is executed *after* the load, as required by the assignment.

6.4.1 d.i – new transaction attributes

Every transaction is extended with a random payment method, promotional offer flag and satisfaction rating. This operation first creates the attribute fields on all transactions, and then another query fills them with values using the same batching pattern as the loader. The attributes are assigned randomly to simulate realistic payment behaviour:

```
MATCH ()-[r:MADE_TRANSACTION]->()
WITH r
CALL {
  WITH r
  SET r.paymentMethod =
    CASE
      WHEN rand() < 0.33 THEN 'Credit Card'
      WHEN rand() < 0.66 THEN 'Mobile Payment'
      ELSE 'PayPal'
    END,
    r.promotionalOffer = rand() < 0.3,
    r.satisfactionRating = 1 + toInteger(rand() * 5)
} IN TRANSACTIONS OF 10000 ROWS;
```



The screenshot shows a query editor with the following SQL code:

```
1 MATCH ()-[r:MADE_TRANSACTION]->()
2 RETURN
3   r.paymentMethod,
4   r.promotionalOffer,
5   r.satisfactionRating
6 LIMIT 10;
```

Below the query editor, there are tabs for 'Table' and 'RAW'. The 'Table' tab is selected, showing a table with 10 rows of data. The columns are 'r.paymentMethod', 'r.promotionalOffer', and 'r.satisfactionRating'.

	r.paymentMethod	r.promotionalOffer	r.satisfactionRating
1	"Credit Card"	true	1
2	"Credit Card"	false	4
3	"PayPal"	true	3
4	"PayPal"	false	4
5	"Credit Card"	true	5
6	"Mobile Payment"	false	3
7	"Credit Card"	false	2
8	"PayPal"	false	1
9	"Mobile Payment"	false	5
10	"Mobile Payment"	false	2

At the bottom of the table, a status message reads: "Started streaming 10 records after 88 ms and completed after 89 ms."

Figure 9: Verification of d.i: ten transactions after the update, each with a payment method, a promotional offer flag and a rating in $[1, 5]$.

6.4.2 d.ii – frequent collaborators

Customers with at least five transactions at the same terminal and average satisfaction ratings within 0.5 of each other are connected by a `FREQUENT_COLLABORATOR` relationship, created with `MERGE` so that it is directly queryable afterwards:

```
MATCH (c1:Customer)-[r1:MADE_TRANSACTION]->(t:Terminal)
MATCH (c2:Customer)-[r2:MADE_TRANSACTION]->(t)
WHERE c1.id < c2.id
WITH c1, c2, t,
      count(r1) AS tx1, count(r2) AS tx2,
      avg(r1.satisfactionRating) AS avgSat1,
      avg(r2.satisfactionRating) AS avgSat2
WHERE tx1 >= 5 AND tx2 >= 5
      AND abs(avgSat1 - avgSat2) <= 0.5
MERGE (c1)-[:FREQUENT_COLLABORATOR {
  terminalId: t.id,
  avgRatingDiff: abs(avgSat1 - avgSat2)
}]->(c2)
RETURN count(*);
```



Figure 10: Operation d.ii, 50 MB: 32 323 relationships created in 7 min 28 s.



Figure 11: Operation d.ii, 100 MB: 33 154 relationships in 30 min 53 s.

```

1 MATCH (c1:Customer)-[r1:MADE_TRANSACTION]->(t:Terminal)
2 MATCH (c2:Customer)-[r2:MADE_TRANSACTION]->(t)
3 WHERE c1.id < c2.id
4 WITH c1,c2,t,
5 count(r1) AS tx1,
6 count(r2) AS tx2,
7 avg(r1.satisfactionRating) AS avgSat1,
8 avg(r2.satisfactionRating) AS avgSat2
9 WHERE tx1 >= 5
10 AND tx2 >= 5
11 AND abs(avgSat1 - avgSat2) <= 0.5
12 MERGE (c1)-[:FREQUENT_COLLABORATOR {
13 terminalId: t.id,
14 avgRatingDiff: abs(avgSat1 - avgSat2)
15 }]->(c2)
16 RETURN count(*);

```

Table RAW

count(*)
33697

Created 33,697 relationships, set 67,394 properties
Started streaming 1 record after 2 ms and completed after 2 hours 47 minutes.

Figure 12: Operation d.ii, 200 MB: 33 697 relationships in 2 h 47 min. The number of relationships barely grows with the dataset size, but the cost of discovering them does.

6.5 Operation e – transactions and outliers per day of the week

For each day of the week, we create a logical grouping of transactions by extracting the day-of-week from each transaction timestamp. This “day-of-week relationship” is implicit in the temporal properties of transactions and is particularly useful for operation e because: (1) it reveals whether fraud patterns vary by day of the week; (2) it allows operational teams to anticipate and prepare for high-risk days; and (3) in the context of fraud detection, certain days may have different customer behaviour, affecting the distribution of transaction amounts and outlier rates. For each day of the week, the total number of transactions and the proportion flagged as outliers (the `isOutlier` mark of operation b) are computed in a single aggregation pass:

```

MATCH ()-[r:MADE_TRANSACTION]->()
WITH
CASE datetime(replace(r.tx_datetime,' ','T')).dayOfWeek
  WHEN 1 THEN 'Monday' WHEN 2 THEN 'Tuesday'
  WHEN 3 THEN 'Wednesday' WHEN 4 THEN 'Thursday'
  WHEN 5 THEN 'Friday' WHEN 6 THEN 'Saturday'
  WHEN 7 THEN 'Sunday'
END AS dayOfWeek,
r.isOutlier AS isOutlier
RETURN
dayOfWeek,
count(*) AS totalTransactions,
sum(CASE WHEN isOutlier = true THEN 1 ELSE 0 END)
  AS outlierTransactions,
100.0 * sum(CASE WHEN isOutlier = true THEN 1 ELSE 0 END)
  / count(*) AS outlierPercentage
ORDER BY dayOfWeek;

```

34 ORDER BY dayOfWeek;

Table RAW

	dayOfWeek	totalTransactions	outlierTransactions	outlierPercentage
1	"Friday"	145551	5204	3.5753790767497304
2	"Monday"	145797	3830	2.6269401976721056
3	"Saturday"	142510	3915	2.747175636797418
4	"Sunday"	146014	3768	2.580574465462216
5	"Thursday"	146079	5042	3.451557034207518
6	"Tuesday"	145410	4956	3.4082937899731793
7	"Wednesday"	144521	5113	3.537894146871389

Started streaming 7 records after 294 ms and completed after 4,650 ms.

Figure 13: Operation e, 50 MB (summary table): about 145 000 transactions per day, outlier proportion between 2.6% and 3.6%, in 4.65 s.

200mb\$ MATCH ()-[r:MADE_TRANSACTION]->() WITH CASE datetime(replace(

Table RAW

	dayOfWeek	totalTransactions	outlierTransactions	outlierPercentage
1	"Friday"	578082	805	0.13925360070024667
2	"Monday"	581064	786	0.13526909256123248
3	"Saturday"	579216	791	0.1365639070743902
4	"Sunday"	583908	789	0.13512402638771862
5	"Thursday"	578656	810	0.13997953879334182
6	"Tuesday"	582248	799	0.1372267487393688
7	"Wednesday"	578220	716	0.12382830064681263

Started streaming 7 records after 211 ms and completed after 40,578 ms.

Figure 14: Operation e, 200 MB: about 580 000 transactions per day in 40.6 s. Thursday has the highest outlier percentage and Wednesday the lowest; the proportions are very stable across the week, as the simulator has no weekly pattern.

7 Performance evaluation

Operation	50 MB	100 MB	200 MB
Load: customers	—	0.212 s	—
Load: terminals	—	0.031 s	—
Load: transactions	—	12.4 s	—
a – shared terminals	4 min 46 s	18 min 36 s	1 h 30 min
b – median outliers	completed	out of memory	out of memory
c – CN_3 (optimised)	0.159 s	n.m.	0.828 s
d.ii – frequent collab.	7 min 28 s	30 min 53 s	2 h 47 min
e – day-of-week summary	4.650 s	8.888 s	40.578 s

Table 3: Execution times on the three datasets (“n.m.” = not measured). The load timings are those reported by the verification script on the 100 MB dataset.

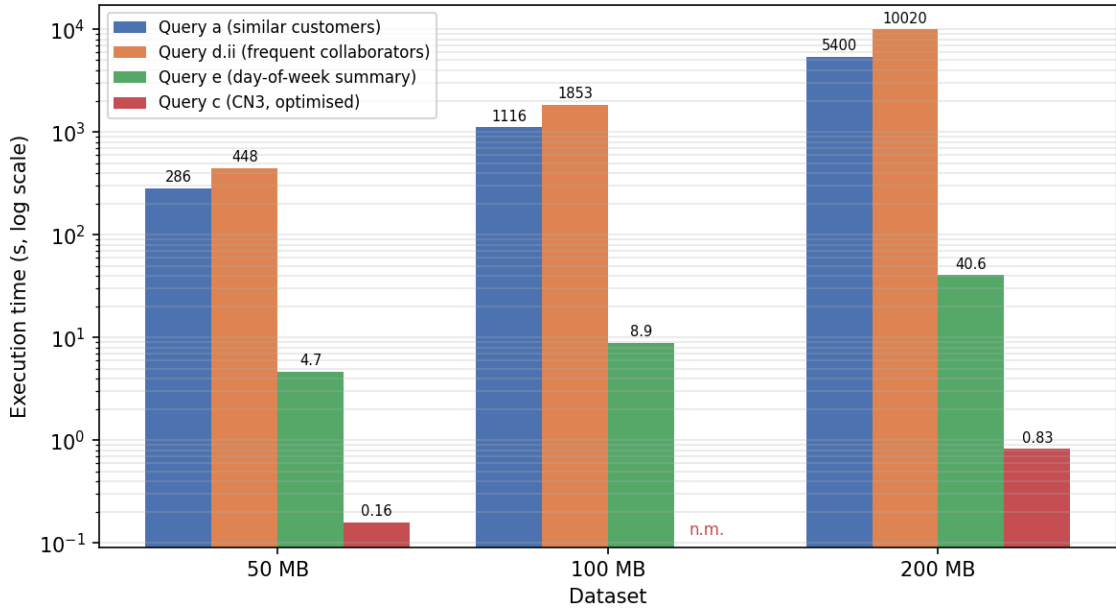


Figure 15: Execution times on the three datasets (logarithmic scale). Operation b is absent because it only completed on the 50 MB dataset.

Loading is linear in the number of rows and dominated by the transaction file; the pre-created indexes (fast `MATCH` by id) and the 10 000-row batching (constant memory) are what make it feasible.

Operation a grows super-linearly: $\approx 3.9\times$ slower from 50 to 100 MB and $\approx 4.8\times$ from 100 to 200 MB, for a $2\times$ data increase each time. The initial pattern produces one row per pair of transactions meeting at a terminal, so doubling the transaction density roughly quadruples the matched paths. The result itself stays tiny (one pair per dataset): most of the work is spent aggregating duplicate paths.

Operation b is memory-bound: the exact median requires collecting and sorting the full list of Q4 amounts of each terminal, which no longer fits in the transaction memory beyond 50 MB.

Operation c stays below one second on every dataset thanks to the stepwise `WITH DISTINCT` deduplication: each expansion works on at most 2000 customers and 300 terminals, regardless of the number of transactions.

Operation d.ii shows a stable output with a growing cost: the relationships only increase from 32 323 to 33 697 (+4%) while the time grows from 7.5 min to 2 h 47 min. With a fixed population the qualifying pairs saturate quickly; the extra transactions only add work, not results.

Operation e scales roughly linearly (4.7 s, 8.9 s, 40.6 s): it is a single scan with a 7-group aggregation; the per-row `datetime()` parsing of the string timestamps is the main cost.

Patterns and possible improvements. The patterns we applied are: index-backed lookups, batched writes (`CALL {} IN TRANSACTIONS`), deduplication with `WITH DISTINCT` (operation c), filtering before the expensive aggregation (operation a), and materialisation of derived data (`isOutlier`, `FREQUENT_COLLABORATOR`). The main improvements suggested by the measurements would be: storing `tx_datetime` as a native datetime (or precomputing year/quarter/day-of-week as properties at load time) to avoid string parsing in operations b and e; computing the median of operation b with `percentileCont`, which streams instead of collecting lists and would avoid the out-of-memory failures; and increasing `dbms.memory.transaction.total.max`.

8 Reproducing the experiments

1. Generate the datasets with `SimulatedDataset.ipynb` (2000 customers, 300 terminals, $r = 5$; 300, 600 and 1200 days) and export the three CSV tables of each run.
2. Create one Neo4j 5.x database per dataset and copy its CSV files into the `import` folder of the instance.
3. Run the loading script (indexes, customers, terminals, transactions, temporal properties), then the scripts of operations a–e. Operation d must run after the load; operation e must run after the outlier-marking variant of operation b.
4. All reported times are the ones displayed by Neo4j (“completed after ...”).